JasperGold Formal Verification

FSM-Based Secure Arithmetic Processing Module

PS-23 | RTL Design and Formal Verification Project

Team Members

B. Surya Ganesh — VLSI Trainee, B.Tech ECE
V. Sri Charan — VLSI Trainee, B.Tech ECE
T.D.L. Manohar — VLSI Trainee, B.Tech ECE
Naga Durga Srinivas — VLSI Trainee, B.Tech ECE
K.L. Kavya Sri — VLSI Trainee, B.Tech ECE
G. Deevana — VLSI Trainee, B.Tech ECE

GitHub: https://github.com/vasanthasricharan/JasperGold-Formal-VRF

Abstract

This project implements and formally verifies a 4-state FSM-based Secure Arithmetic Processing Module using SystemVerilog (SVA) and Cadence JasperGold Formal Verification. The design processes ADD, SUB (signed-safe), AND, and XOR arithmetic/logic operations with security and power-control gating. Formal verification mathematically proves design correctness by exhaustively exploring all reachable states and input combinations, going far beyond what directed simulation can achieve.

A total of 24 assertion properties were proven and 11 cover points were confirmed reachable using BMC, K-Induction, and ABC proof engines in JasperGold 2022.09. No counterexamples were generated.

Problem Statement

Simulation-based verification tests only a limited, pre-selected set of input scenarios. As a result, corner-case bugs, illegal FSM states, security violations under rare input combinations, and deadlock conditions may remain undetected. The Universal Verification Methodology (UVM), while powerful for constrained-random testing, cannot exhaustively cover all 2^N input combinations for even a modest N-bit input space.

Project Objective

The objective is to demonstrate the advantages of formal verification over simulation by proving RTL correctness mathematically. Specific goals include:

- Prove all FSM state transitions are correct and complete
- Verify output signals (grant, valid, error_flag, result) are correct in every state
- Confirm the design is deadlock-free (no state from which recovery is impossible)
- Prove all 4 arithmetic operations produce correct results for all 4-bit input values
- Confirm that security and power failure conditions trigger ERROR recovery correctly

- Confirm all states are reachable (no dead states)

Design Overview

Module Specification

Module name: `jaspergold_complete_design` | **Parameters:** `DATA_W=4 (default), RES_W=DATA_W+1=5`

Signal	Direction	Width	Description
clk	Input	1	System clock (posedge active)
rst	Input	1	Active-HIGH synchronous reset
req	Input	1	Request to start operation (1-cycle pulse)
data_a	Input	DATA_W	Operand A
data_b	Input	DATA_W	Operand B
opcode	Input	2	00=ADD, 01=SUB, 10=AND, 11=XOR
secure_access	Input	1	Security gate — LOW triggers ERROR
power_enable	Input	1	Power gate — LOW triggers ERROR
grant	Output	1	Asserted when FSM is in PROCESS state
result	Output	RES_W	Operation result (extra bit for carry/sign)
valid	Output	1	Asserted when FSM is in DONE state
error_flag	Output	1	Asserted in PROCESS (fault) or ERROR state

FSM State Encoding

State	Encoding	Description
IDLE	2'b00	Waiting for request signal (req). Default state after reset.
PROCESS	2'b01	Executing requested arithmetic operation. grant is asserted.
DONE	2'b10	Operation completed successfully. valid is asserted for one cycle.
ERROR	2'b11	Security or power failure detected. error_flag is asserted.

FSM Transition Rules

- IDLE → PROCESS: when req is asserted (one-cycle pulse)
- PROCESS → DONE: when power_enable = 1 AND secure_access = 1
- PROCESS → ERROR: when power_enable = 0 OR secure_access = 0
- DONE → IDLE: unconditional (one-cycle result hold, then auto-return)
- ERROR → IDLE: unconditional (one-cycle error hold, then auto-recovery)
- Reset (rst=1): forces FSM to IDLE on the next posedge clock

Arithmetic Operations

Opcode	Operation	Result Expression	Notes
2'b00	ADD	{1'b0, data_a} + {1'b0, data_b}	Unsigned addition. Carry captured in MSB of RES_W result.
2'b01	SUB	$signed({1'b0,a}) - $signed({1'b0,b})	Signed-safe. Prevents unsigned underflow truncation (e.g., 0-15 = -15 represented correctly).
2'b10	AND	{1'b0, data_a & data_b}	Bitwise AND, zero-extended to RES_W.
2'b11	XOR	{1'b0, data_a ^ data_b}	Bitwise XOR, zero-extended to RES_W.

Verification Methodology

This project uses a two-track verification approach: directed RTL simulation and formal property verification. Both tracks share the same DUT (FSM_arithmetic_model.v) and target the same correctness criteria.

Track 1: RTL Simulation

Two testbenches were developed:

- FSM_arithmetic_model_normal_tb.v — Directed testbench covering all 4 opcodes, security violations, power failures, boundary values, and X-state injection. Produces an analytics report with pass/fail counts.
- FSM_arithmetic_model_jasper_tb.v — Exhaustive simulation testbench running all 4 opcodes × all 256 combinations of 4-bit data inputs (1024 operations total) with inline SVA assertion checking.

Track 2: JasperGold Formal Verification

The formal flow uses two files: the RTL DUT and a standalone SVA property module bound non-invasively.

- FSM_arithmetic_model.v — RTL DUT with inline SVA assertions inside an `ifdef FORMAL guard (compiled only by JasperGold).
- FSM_arithmetic_properties.sv — Standalone property module bound via SystemVerilog bind. Contains 17 assertions, 7 cover points, and 3 assumptions. Non-invasive: the DUT source is not modified.
- jg_run.tcl — TCL automation script driving analyze → elaborate → clock/reset → assume → prove → cover → report.

Input Assumptions (Formal Engine Constraints)

Assumption	Constraint	Rationale
ASSUME_REQ_PULSE	req |=> !req	req must de-assert after one cycle per protocol spec
ASSUME_DATA_STABLE_IN_PROCESS	$stable(data_a, data_b, opcode) in PROCESS	Inputs must not change mid-operation
ASSUME_CTRL_STABLE_IN_PROCESS	$stable(power_enable, secure_access) in PROCESS	Control signals must not change mid-operation

Formal Verification Concept

Formal verification uses mathematical techniques (Boolean satisfiability, model checking, abstract interpretation) to exhaustively prove or disprove properties about a hardware design for all possible input sequences. Unlike simulation, which samples behavior at specific time points with specific inputs, formal verification reasons over the entire state space.

Proof Engines Used

- BMC (Bounded Model Checking): Unrolls the design for k steps and searches for property violations. Fast for finding bugs at low depth. Used for k=1 and k=2 properties in this project.
- K-Induction: Proves properties hold for all k and beyond by combining a base case (BMC) with an inductive step. Used for FSM transition properties requiring k=2 and k=3 depths.
- ABC (Combinational Reasoning): Applies AIG-based combinational simplification. Corroborates single-cycle (purely combinational) properties.

SVA Property Types

Type	Keyword	Meaning
Assertion	assert property	Property that must hold for all reachable states
Assumption	assume property	Constraint on inputs — limits engine's exploration space
Cover	cover property	Reachability check — proves a sequence is achievable

Properties Verified

FSM Transition Properties (P1–P7)

ID	Property Name	Assertion	Proven	Engine	Depth
P1	IDLE_TO_PROCESS	(IDLE && req) |=> PROCESS	Yes	K-Ind	k=3
P2	PROCESS_TO_DONE	(PROCESS && pwr && sec) |=> DONE	Yes	K-Ind	k=3
P3	PROCESS_TO_ERROR_PWR	(PROCESS && !power_enable) |=> ERROR	Yes	K-Ind	k=3
P4	PROCESS_TO_ERROR_SEC	(PROCESS && !secure_access) |=> ERROR	Yes	K-Ind	k=3
P5	DONE_TO_IDLE	DONE |=> IDLE (unconditional)	Yes	K-Ind	k=2
P6	ERROR_TO_IDLE	ERROR |=> IDLE (unconditional)	Yes	K-Ind	k=2
P7	RESET_IDLE	$rose(rst) |=> IDLE	Yes	BMC	k=2

Output Signal Correctness (P8–P13)

ID	Property Name	Assertion	Proven	Engine
P8	GRANT_IN_PROCESS	PROCESS |-> grant	Yes	BMC k=1
P8b	NO_GRANT_OUTSIDE_PROCESS	!PROCESS |-> !grant	Yes	BMC k=1
P9	VALID_IN_DONE	DONE |-> valid	Yes	BMC k=1
P9b	NO_VALID_OUTSIDE_DONE	!DONE |-> !valid	Yes	BMC k=1
P10	ERROR_FLAG_INSECURE	(PROCESS && !secure_access) |-> error_flag	Yes	BMC k=1
P11	ERROR_FLAG_POWER	(PROCESS && !power_enable) |-> error_flag	Yes	BMC k=1
P12	ERROR_FLAG_IN_ERROR	ERROR |-> error_flag	Yes	BMC k=1
P13	NO_ERROR_IN_IDLE	IDLE |-> !error_flag	Yes	BMC k=1

Arithmetic Correctness (P14–P17)

ID	Property Name	Verified Condition	Proven	Engine
P14	ADD_CORRECT	result == {1'b0, data_a} + {1'b0, data_b}	Yes	BMC k=1
P15	SUB_CORRECT	result == $signed({1'b0,a}) - $signed({1'b0,b})	Yes	BMC k=1
P16	AND_CORRECT	result == {1'b0, data_a & data_b}	Yes	BMC k=1
P17	XOR_CORRECT	result == {1'b0, data_a ^ data_b}	Yes	BMC k=1

Note: All 24 assertions proven (0 falsified, 0 vacuous, 0 inconclusive). No counterexamples generated.

Cover Points (Reachability Analysis)

Cover Point	Description	Witness Length	Result
COVER_IDLE	IDLE state is reachable	1 cycle	Covered
COVER_PROCESS	PROCESS state is reachable	2 cycles	Covered
COVER_DONE	DONE state is reachable	3 cycles	Covered
COVER_ERROR	ERROR state is reachable	3 cycles	Covered
COVER_HAPPY_PATH	IDLE → PROCESS → DONE → IDLE full round-trip	4 cycles	Covered
COVER_SEC_ERROR_PATH	PROCESS (insecure) → ERROR → IDLE recovery	4 cycles	Covered
COVER_PWR_ERROR_PATH	PROCESS (!power) → ERROR → IDLE recovery	4 cycles	Covered

All 11 cover points reached. All 4 FSM states confirmed reachable. No dead states detected.

JasperGold Session Summary

Parameter	Value
Design	jaspergold_complete_design (DATA_W=4, RES_W=5)
JasperGold Version	2022.09p001
Run Date	2024-11-15, 14:30–14:32
Total Assertions	24
Proven	24 (100%)
Falsified	0
Vacuous	0
Inconclusive	0
Total Cover Points	11
Covered	11 (100%)
Proof Engines	BMC, K-Induction, ABC (parallel, -bg)
Maximum Proof Depth	k = 3
Session Time	13 seconds
Peak Memory	312 MB
Counterexample Files	None (all properties proven)

Simulation Results

Normal Testbench (FSM_arithmetic_model_normal_tb.v)

Directed testbench covering all 4 opcodes, security violations, power failures, and boundary values. Simulator: Icarus Verilog 2012.

Category	Tests Run	Passed	Accuracy
Functional Operations	12	12	100%
Protocol Handshakes	12	12	100%
Security Tests	2	2	100%
Overall Confidence			100%

Exhaustive Testbench (FSM_arithmetic_model_jasper_tb.v)

Exhaustive simulation covering all 4 opcodes × all 4-bit data combinations, plus security, power, and X-state tests. Inline SVA assertions active during simulation.

Category	Count	Result
Total Operations Tested	256	100% Pass

Functional Accuracy	256/256	100%
Security Integrity	All cases	100%
SVA Assertion Failures	0	No failures
Overall Confidence		100%

Simulation vs. Formal Verification Comparison

Criterion	RTL Simulation	JasperGold Formal Verification
Coverage	Selected test cases only	All reachable states and inputs
Corner-case detection	Depends on testbench completeness	Exhaustive — finds all corner cases
Proof type	Test-pass/fail per run	Mathematical proof for all inputs
Deadlock detection	Only if test exercises the path	Proven unconditionally (P5, P6)
State reachability	States visited depend on stimulus	All 4 states confirmed reachable
Security verification	Only tested combinations	All secure_access/power conditions proven
Arithmetic correctness	Sampled values	Proven for all 4-bit input combinations
Runtime	Fast (seconds)	13 seconds for full proof (24 properties)
Counterexample output	Simulation waveform	VCD counterexample file (if any failure)

Key Implementation Notes

Signed Subtraction Correctness

The SUB operation uses sign-extended operands to prevent unsigned underflow truncation. Without sign extension, subtracting a larger value from a smaller one would wrap incorrectly.

```
wire signed [DATA_W:0] signed_a = {1'b0, data_a};
wire signed [DATA_W:0] signed_b = {1'b0, data_b};
// Example: 0 - 15 correctly produces 5'b10001 = -15 (signed)
result = signed_a - signed_b;
```

Latch-Free Output Logic

All output registers (grant, valid, result, error_flag) are assigned safe defaults at the top of the combinational always block before any case logic. This prevents synthesis tools from inferring unintended latches.

Dual-Layer SVA Strategy

Assertions are placed in two locations:

- Inline in FSM_arithmetic_model.v inside `ifdef FORMAL — compiled by JasperGold with +define+FORMAL; ignored by simulation tools
- Standalone in FSM_arithmetic_properties.sv — bound via bind, non-invasive, adds coverage and assumptions not present in the RTL file

TCL Script Hierarchical Path Fix (v4.0)

JasperGold TCL assume statements reference prop_inst.current_state (the bound property module's port) rather than dut.current_state. In batch mode, the design top is jaspergold_complete_design — a path named dut.current_state does not resolve. Using the bound instance path guarantees correct resolution in both batch and GUI modes.

Advantages of JasperGold Formal Verification

- Exhaustive state-space exploration — no blind spots from missing test cases
- Mathematical proof — properties are proven, not just sampled
- Deadlock and liveness verification — P5 and P6 prove auto-recovery from DONE and ERROR
- Reachability analysis — all FSM states confirmed accessible; no dead states
- Bug-finding counterexamples — if a property fails, an exact reproduction trace is generated as VCD
- Fast runtime — complete 24-property proof in 13 seconds
- Scalable parameterization — DATA_W can be changed at elaborate time without modifying RTL

Conclusion

This project successfully implemented and formally verified a 4-state FSM-based Secure Arithmetic Processing Module. All 24 SVA assertion properties were proven across FSM transitions, output signal correctness, reset behavior, and arithmetic correctness for ADD, SUB (signed-safe), AND, and XOR operations. All 11 cover points were reached, confirming that all four FSM states are reachable and both error recovery paths (security violation and power failure) are exercisable.

The comparison between RTL simulation and formal verification confirms that while simulation validates expected behavior on chosen test vectors, formal verification provides mathematical certainty over all possible input combinations and state transitions. The project demonstrates a complete, professional-grade formal verification flow using JasperGold with bind-based non-invasive property attachment, parallel proof engines, and automated report generation.

Future Enhancements

The following extensions are direct, natural continuations of the work completed in this project:

- Parameterized width sweep: The design already supports a DATA_W parameter. A regression script that runs the full proof for DATA_W = 4, 8, 16, and 32 would confirm that no arithmetic overflow or FSM correctness issue is introduced at wider data paths.
- Extended arithmetic operations: Adding MUL (multiply) or shift operations to the ALU and authoring corresponding SVA properties (P18, P19) would test the formal flow on non-trivial combinational arithmetic.
- Multi-cycle PROCESS state: Modifying the PROCESS state to take N clock cycles (e.g., for a pipelined multiplier) would require temporal SVA properties with ##N operators, extending the current single-cycle proof methodology.

- Concurrent request handling: Adding a FIFO or request queue and verifying ordering and fairness properties with SVA sequences would extend the protocol verification capability demonstrated in this project.
- Security property strengthening: Adding information-flow properties (e.g., result must be 0 whenever secure_access=0 regardless of opcode) would strengthen the existing P10–P13 security assertions with confidentiality-style guarantees.
- Transition coverage closure: Adding toggle and FSM transition cover points beyond the 7 currently implemented to achieve 100% transition coverage in the cover report.

References

- Cadence JasperGold FPV Documentation — https://www.cadence.com/jasper-gold-verification-platform
- IEEE Std 1800-2012 — SystemVerilog Language Reference Manual (SVA concurrent assertions)
- RTL source: FSM_arithmetic_model.v — jaspergold_complete_design, Version 3.0
- Property module: FSM_arithmetic_properties.sv — Version 3.0 (17 assertions, 7 cover points, 3 assumptions)
- TCL script: jg_run.tcl — Version 4.0 (bind-path fix, parallel engines, CEX dump)
- JasperGold proof results: jg_assert_report.txt, jg_cover_report.txt, jg_full_session_log.txt